

Milestone 3: Software Architecture Analysis for StreamFlix

Introduction

StreamFlix is our video streaming platform that handles users, content (movies and TV series), personalized recommendations, watch history, ratings, and notifications. For this milestone, we're analyzing three different architectural approaches to figure out which one makes the most sense for our system.

Architecture 1: Monolithic Architecture

Description

In a monolithic setup, everything lives in one application. All of StreamFlix's functionality—user management, content catalog, video streaming, recommendations, analytics—is bundled together in a single codebase with one shared database.

Main Components:

- **Presentation Layer:** REST controllers (UserController, ContentController, VideoController, RecommendationController)
- **Business Logic Layer:** Service classes (UserService, ContentService, VideoService, RecommendationService)
- **Data Access Layer:** JDBC templates and our domain models (User, Content, Rating, WatchHistory)
- **Shared Resources:** Singleton instances like DatabaseConnectionPool, CacheManager, ConfigurationManager

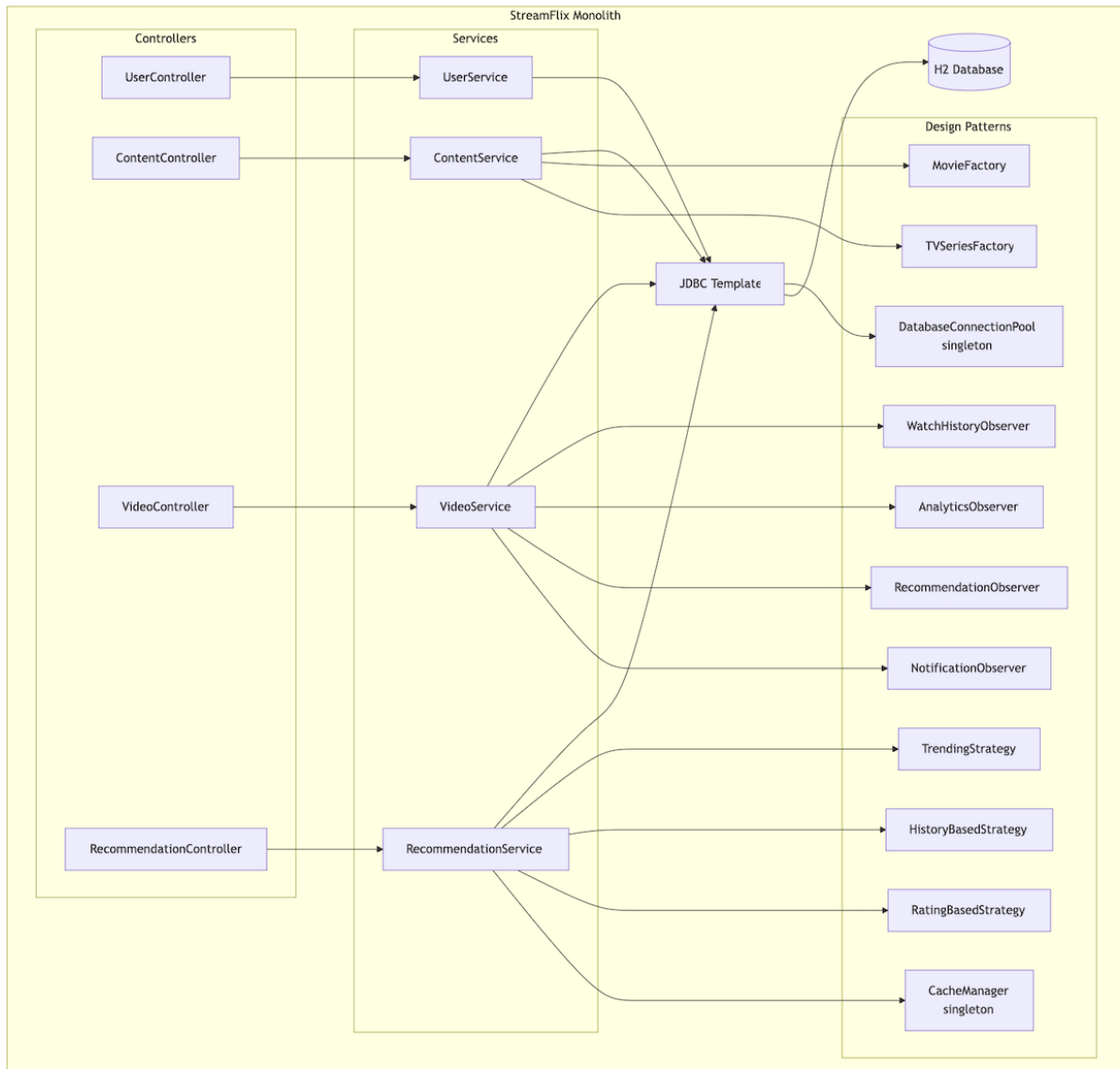
Interactions:

All user requests go through our single Spring Boot application. Controllers call services, which then access the H2 in-memory database. We use the Observer pattern to notify multiple handlers (WatchHistoryObserver, AnalyticsObserver, RecommendationObserver, NotificationObserver) when stuff happens. The Strategy pattern (TrendingStrategy, HistoryBasedStrategy, RatingBasedStrategy) lets us switch recommendation algorithms at runtime.

Data Flow:

1. Client sends HTTP request to REST endpoint
2. Controller passes it to the right service
3. Service runs business logic using our shared singletons
4. Data gets saved or retrieved from the single H2 database
5. Events fire off to observers for things like analytics and notifications
6. Response goes back to client

Component Diagram



graph TB

subgraph mono[StreamFlix Monolith]

subgraph controllers[Controllers]

uc[UserController]

cc[ContentController]

vc[VideoController]

rc[RecommendationController]

end

subgraph services[Services]

us[UserService]

cs[ContentService]

vs[VideoService]

rs[RecommendationService]

end

subgraph patterns[Design Patterns]

mf[MovieFactory]

tf[TVSeriesFactory]

ts[TrendingStrategy]

hbs[HistoryBasedStrategy]

rbs[RatingBasedStrategy]

who[WatchHistoryObserver]

ao[AnalyticsObserver]

ro[RecommendationObserver]

no[NotificationObserver]

pool[DatabaseConnectionPool
singleton]

cache[CacheManager
singleton]

end

jdbc[JDBC Template]

db[(H2 Database)]

end

uc --> us

cc --> cs

vc --> vs

rc --> rs

us --> jdbc

cs --> jdbc

vs --> jdbc

rs --> jdbc

cs --> mf

cs --> tf

rs --> ts

rs --> hbs

rs --> rbs

vs --> who

vs --> ao

vs --> ro

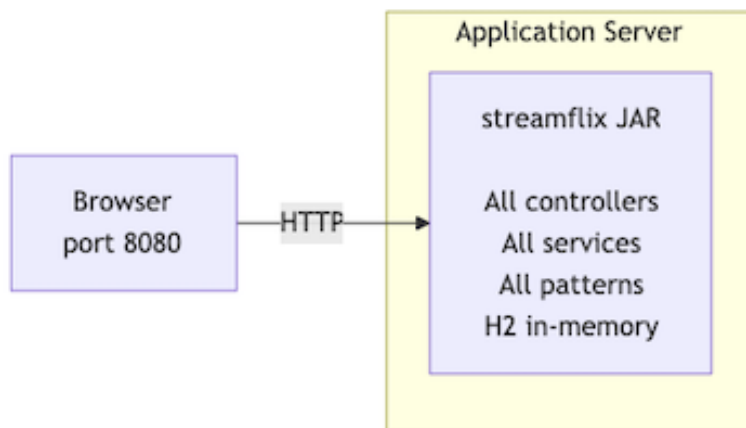
vs --> no

jdbc --> pool

jdbc --> db

rs --> cache

Deployment Diagram



graph LR

client[Browser
port 8080]

subgraph server[Application Server]

jar[streamflix JAR

All controllers

All services

All patterns
H2 in-memory]

end

client -->|HTTP| jar

Pros

- **Simplicity:** One codebase makes development and debugging straightforward. When something breaks, you know exactly where to look.
- **Easy Deployment:** Just one JAR file (our pom.xml builds streamflix-1.0.0-SNAPSHOT.jar) and we're done.
- **Performance:** No network delays between components since everything's in-process. Method calls are fast.
- **Transaction Management:** ACID transactions work across all operations with our single database, so we don't have to worry about distributed transaction headaches.
- **Testing:** Integration testing is simple—we already have HelloControllerTest working.
- **Design Pattern Integration:** All four patterns (Factory, Strategy, Observer, Singleton) work together without any issues.
- **Development Speed:** For our team of 3, we can iterate really fast. The current POC shows how quickly we can prototype features.

Cons

- **Scalability Issues:** Can't scale individual parts independently. If recommendations need more resources, we have to scale the whole app.
- **Single Point of Failure:** If the application crashes, everything goes down—users can't browse, watch, or do anything.
- **Technology Lock-in:** We're stuck with Spring Boot 1.2.7 and Java 8 for everything. Can't use newer tech for specific features.
- **Deployment Risk:** Every change means redeploying the entire application, which is risky for production.
- **Team Coordination:** As our team grows, everyone working in the same codebase will create merge conflicts.
- **Memory Constraints:** H2 in-memory database (from our application.properties) limits how much data we can store.

- **Video Streaming Limitations:** Actually streaming video files would need major architectural changes—right now we're just simulating events.
-

Architecture 2: Microservices Architecture

Description

With microservices, we'd break StreamFlix into independent services that each handle one specific domain. Each service owns its data and talks to other services through REST APIs or message queues.

Main Components:

- **User Service:** Handles registration, authentication, and user profiles
- **Content Service:** Manages the movie/TV series catalog using our Factory pattern
- **Video Service:** Tracks watch events, ratings, and fires off Observer notifications
- **Recommendation Service:** Uses our Strategy pattern for personalized recommendations
- **Analytics Service:** Processes viewing data and updates metrics
- **Notification Service:** Sends user notifications asynchronously
- **API Gateway:** Routes requests to the right services and handles authentication

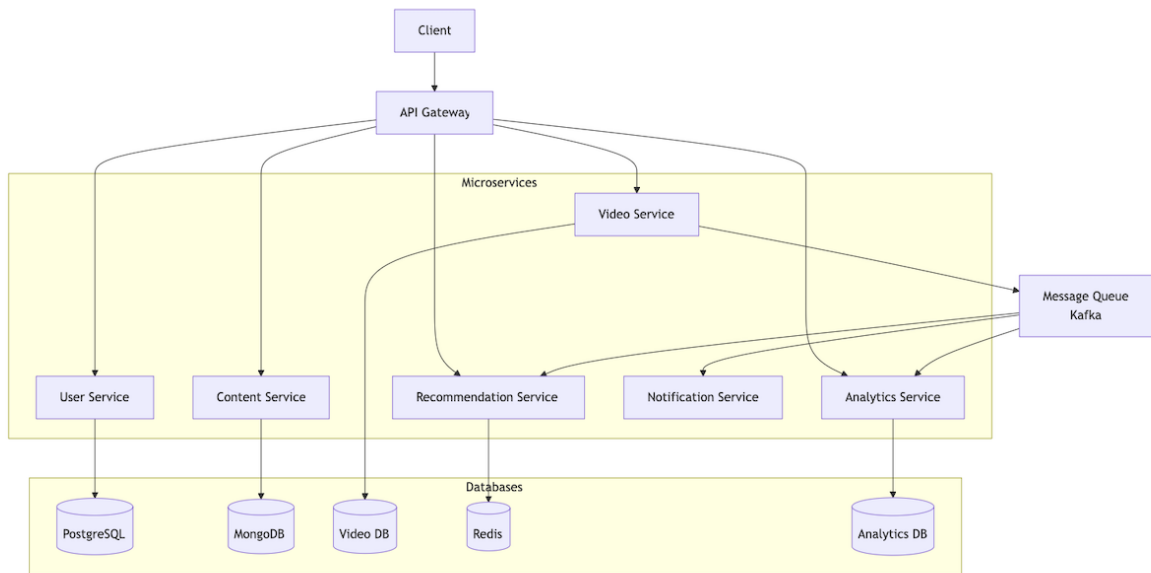
Interactions:

Each microservice has its own REST endpoints. Services talk to each other asynchronously through a message queue (RabbitMQ or Kafka) for events. The API Gateway is the single entry point. Each service gets its own database (polyglot persistence).

Data Flow:

1. Client request hits the API Gateway
2. Gateway routes to the appropriate microservice
3. Service processes the request using its local database
4. For operations involving multiple services, the service publishes events to the message queue
5. Other interested services consume these events asynchronously
6. Response gets aggregated and sent back to the client

Component Diagram



graph TB

client[Client]

gw[API Gateway]

subgraph services[Microservices]

us[User Service]

cs[Content Service]

vs[Video Service]

rs[Recommendation Service]

as[Analytics Service]

ns[Notification Service]

end

subgraph databases[Databases]

userdb[(PostgreSQL)]

contentdb[(MongoDB)]

videodb[(Video DB)]

rediscache[(Redis)]

analyticsdb[(Analytics DB)]

end

mq[Message Queue
Kafka]

client --> gw

gw --> us

gw --> cs

gw --> vs

gw --> rs

gw --> as

us --> userdb

cs --> contentdb

vs --> videodb

rs --> rediscache

as --> analyticsdb

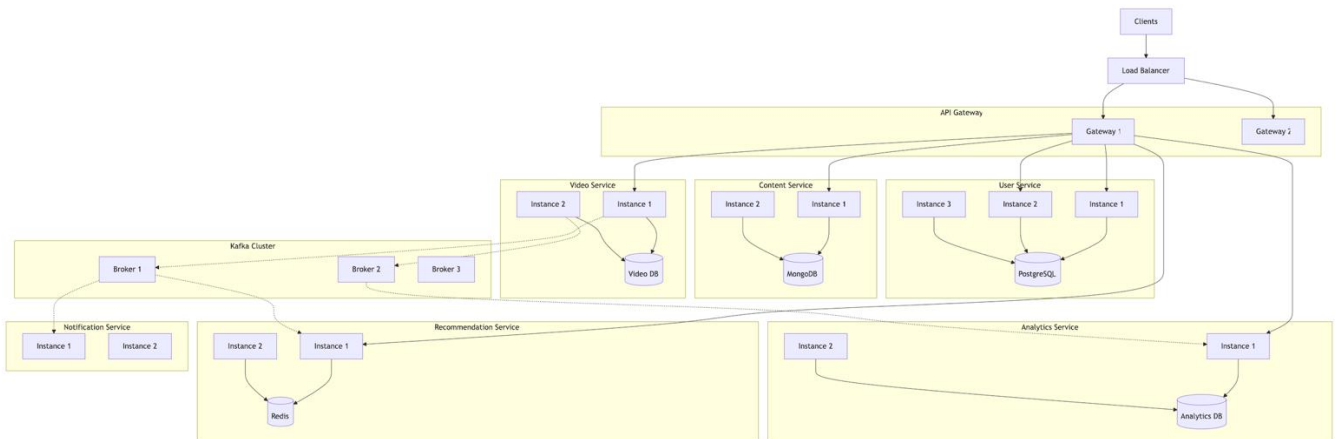
vs --> mq

mq --> as

mq --> rs

mq --> ns

Deployment Diagram



graph TB

clients[Clients]

lb[Load Balancer]

subgraph gateway[API Gateway]

gw1[Gateway 1]

gw2[Gateway 2]

end

subgraph userservice[User Service]

us1[Instance 1]

us2[Instance 2]

us3[Instance 3]

uspg[(PostgreSQL)]

end

subgraph contentservice[Content Service]

cs1[Instance 1]

cs2[Instance 2]

csmongo[(MongoDB)]

end

subgraph videoservice[Video Service]

vs1[Instance 1]

vs2[Instance 2]

vsdb[(Video DB)]

end

subgraph recoservice[Recommendation Service]

rs1[Instance 1]

rs2[Instance 2]

redis[(Redis)]

end

subgraph analyticsservice[Analytics Service]

as1[Instance 1]

as2[Instance 2]

asdb[(Analytics DB)]

end

```
subgraph notifservice[Notification Service]
```

```
    ns1[Instance 1]
```

```
    ns2[Instance 2]
```

```
end
```

```
subgraph kafka[Kafka Cluster]
```

```
    k1[Broker 1]
```

```
    k2[Broker 2]
```

```
    k3[Broker 3]
```

```
end
```

```
clients --> lb
```

```
lb --> gw1
```

```
lb --> gw2
```

```
gw1 --> us1
```

```
gw1 --> us2
```

```
gw1 --> cs1
```

```
gw1 --> vs1
```

```
gw1 --> rs1
```

```
gw1 --> as1
```

```
us1 --> uspg
```

```
us2 --> uspg
```

us3 --> uspg

cs1 --> csmongo

cs2 --> csmongo

vs1 --> vsdb

vs2 --> vsdb

rs1 --> redis

rs2 --> redis

as1 --> asdb

as2 --> asdb

vs1 -.-> k1

vs2 -.-> k2

k1 -.-> ns1

k2 -.-> as1

k1 -.-> rs1

Pros

- **Independent Scalability:** We could scale just the recommendation service during peak hours without touching the video service.
- **Technology Flexibility:** Could use MongoDB for content catalog, PostgreSQL for user data, Redis for recommendation cache—whatever fits best.
- **Fault Isolation:** If the notification service crashes, users can still watch videos. That's huge for reliability.
- **Team Autonomy:** Different team members can own different services. Elena could work on User Service, Miruna on Content, Daria on Video/Streaming.
- **Continuous Deployment:** Can deploy recommendation updates without affecting video playback.
- **Design Pattern Preservation:** Our Factory, Strategy, Observer, and Singleton patterns still work within each service.
- **Better Resource Utilization:** Analytics service could do batch processing while video service optimizes for real-time streaming.

- **Matches Industry Standards:** Netflix, Disney+, and other major streaming platforms use microservices for exactly these reasons.

Cons

- **Complexity Jumps:** Managing 6+ services instead of our current single App.java file. That's a lot more moving parts.
- **Distributed Transactions:** What happens if a user registers but creating their recommendation profile fails? We'd need sagas or other patterns.
- **Network Latency:** Every inter-service call adds overhead. Not as fast as in-process method calls.
- **Operational Overhead:** We'd need service discovery, distributed logging, monitoring. Our simple single-JAR deployment would be gone.
- **Data Duplication:** Services might cache user data, leading to sync issues.
- **Testing Gets Hard:** Integration tests would require spinning up multiple services—way more complex than our current HelloControllerTest.
- **Debugging Challenges:** Tracing a request across 5 services is harder than debugging a single stack trace.
- **Infrastructure Costs:** Need Kubernetes for orchestration, message queues, multiple databases, adds up fast.

Architecture 3: Event-Driven Architecture (Distributed)

Description

Event-driven architecture structures StreamFlix around events. Components communicate by publishing and subscribing to domain events through an event broker instead of calling each other directly.

Main Components:

- **Event Broker:** Central message bus (Apache Kafka or AWS EventBridge)
- **User Management Domain:** Publishes UserRegistered, UserSubscriptionChanged events
- **Content Domain:** Publishes ContentAdded, ContentUpdated events
- **Video Playback Domain:** Publishes VideoWatched, ContentRated events (similar to our current VideoWatchedEvent, ContentRatedEvent)
- **Recommendation Engine:** Consumes watch/rating events, publishes RecommendationGenerated
- **Analytics Processor:** Consumes all events, updates view counts and ratings (like our AnalyticsObserver)
- **Notification Service:** Consumes events and sends alerts (like our NotificationObserver)

- **Event Store:** Persists all events for replay and auditing

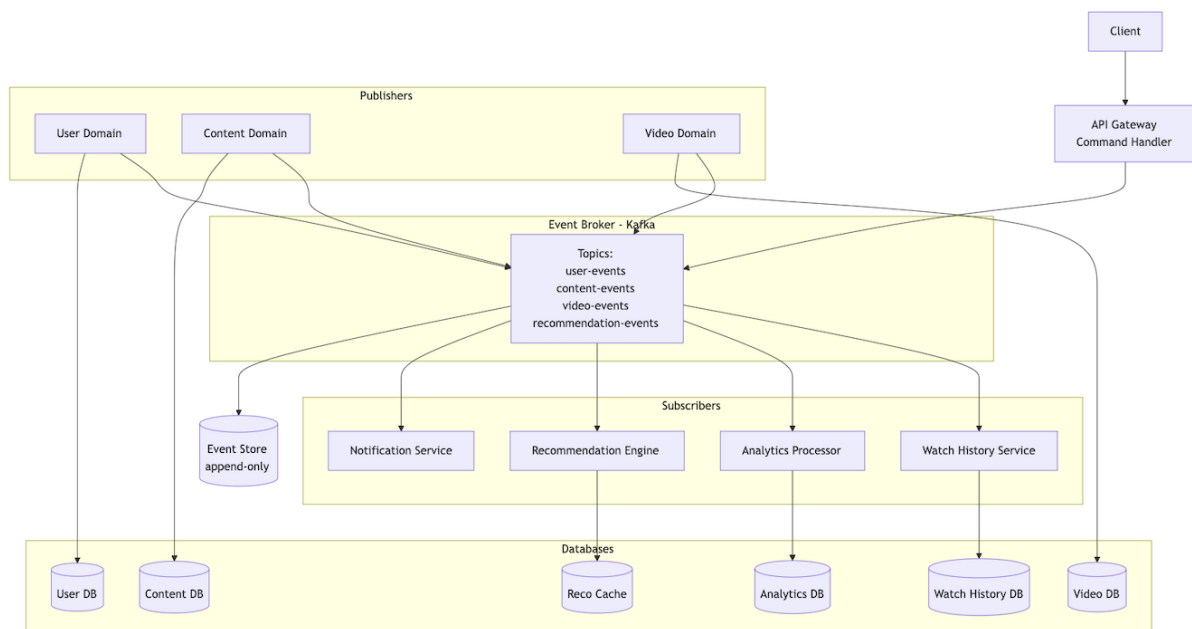
Interactions:

Domains publish events to specific topics (user-events, video-events, etc.). Subscribers consume events asynchronously. There's no direct service-to-service calls: everything's loosely coupled through events. Event sourcing lets us rebuild state from event history.

Data Flow:

1. User action triggers a command (e.g., watch video)
2. Video domain validates and publishes VideoWatched event
3. Event broker routes the event to all subscribers
4. Watch History service updates the user's history
5. Analytics service increments view count
6. Recommendation service recalculates suggestions
7. Notification service sends achievement notification
8. All these operations happen asynchronously and independently

Component Diagram



graph TB

client[Client]

gw[API Gateway
Command Handler]

subgraph broker[Event Broker - Kafka]

topics[Topics:
user-events
content-events
video-events
recommendation-events]

end

subgraph publishers[Publishers]

ud[User Domain]

cd[Content Domain]

vd[Video Domain]

end

subgraph subscribers[Subscribers]

wh[Watch History Service]

ap[Analytics Processor]

re[Recommendation Engine]

ns[Notification Service]

end

subgraph dbs[Databases]

userdb[(User DB)]

contentdb[(Content DB)]

videodb[(Video DB)]

whdb[(Watch History DB)]

analyticsdb[(Analytics DB)]

recocache[(Reco Cache)]

end

eventstore[(Event Store
append-only)]

client --> gw

gw --> topics

ud --> topics

cd --> topics

vd --> topics

topics --> wh

topics --> ap

topics --> re

topics --> ns

topics --> eventstore

ud --> userdb

cd --> contentdb

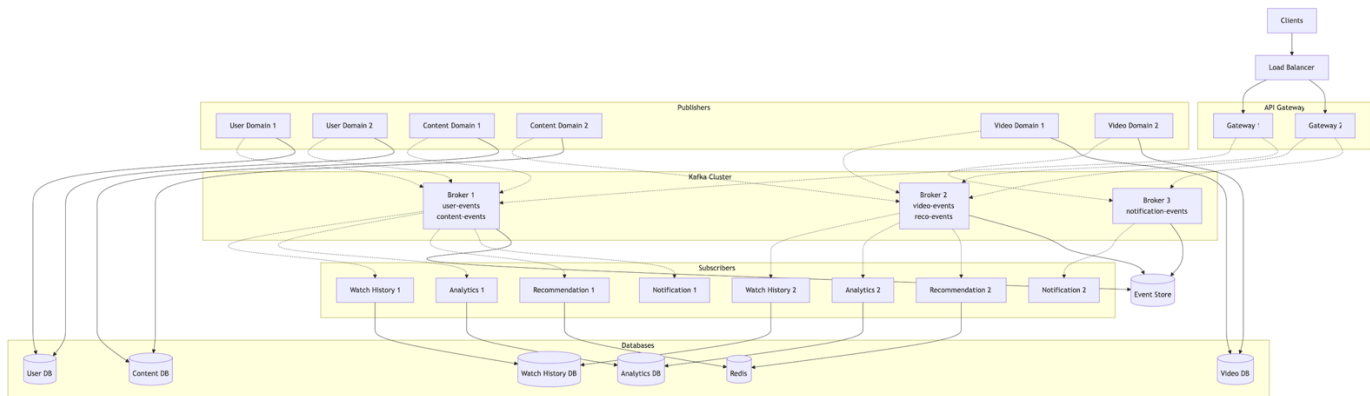
vd --> videodb

wh --> whdb

ap --> analyticsdb

re --> recocache

Deployment Diagram



graph TB

clients[Clients]

lb[Load Balancer]

subgraph apigateway[API Gateway]

gw1[Gateway 1]

gw2[Gateway 2]

end

subgraph kafkacluster[Kafka Cluster]

k1[Broker 1
user-events
content-events]

k2[Broker 2
video-events
reco-events]

k3[Broker 3
notification-events]

end

subgraph pubs[Publishers]

ud1[User Domain 1]

ud2[User Domain 2]

cd1[Content Domain 1]

cd2[Content Domain 2]

vd1[Video Domain 1]

vd2[Video Domain 2]

end

subgraph subs[Subscribers]

wh1[Watch History 1]

wh2[Watch History 2]

ap1[Analytics 1]

ap2[Analytics 2]

re1[Recommendation 1]

re2[Recommendation 2]

ns1[Notification 1]

ns2[Notification 2]

end

subgraph storage[Databases]

userdb[(User DB)]

contentdb[(Content DB)]

videodb[(Video DB)]

whdb[(Watch History DB)]

analyticsdb[(Analytics DB)]

redis[(Redis)]

end

eventstore[(Event Store)]

clients --> lb

lb --> gw1

lb --> gw2

gw1 -.-> k1

gw1 -.-> k2

gw2 -.-> k2

gw2 -.-> k3

ud1 -.-> k1

ud2 -.-> k1

cd1 -.-> k1

cd2 -.-> k2

vd1 -.-> k2

vd2 -.-> k3

k1 -.-> wh1

k2 --> wh2

k1 --> ap1

k2 --> ap2

k1 --> re1

k2 --> re2

k1 --> ns1

k3 --> ns2

k1 --> eventstore

k2 --> eventstore

k3 --> eventstore

ud1 --> userdb

ud2 --> userdb

cd1 --> contentdb

cd2 --> contentdb

vd1 --> videodb

vd2 --> videodb

wh1 --> whdb

wh2 --> whdb

ap1 --> analyticsdb

ap2 --> analyticsdb

re1 --> redis

re2 --> redis

Pros

- **Loose Coupling:** Services don't need to know about each other. Our current VideoService observer pattern basically extends to this naturally.
- **Asynchronous Processing:** Non-blocking operations make the system more responsive.
- **Scalability:** Can scale consumers independently based on event volume. If recommendations get heavy, scale just that.
- **Auditability:** Event store gives us complete history—super useful for GDPR compliance and analytics.
- **Resilience:** If a consumer fails, it can replay events without data loss. Nothing gets dropped.
- **Temporal Decoupling:** Producers and consumers don't need to be online at the same time.
- **Design Pattern Fit:** Our Observer pattern in the observer/ package maps directly to this event-driven model.
- **Extensibility:** Want to add email recommendations? Just create a new subscriber. Don't have to modify existing code.

Cons

- **Eventual Consistency:** User rates content, but the analytics dashboard might update a few seconds later. Not immediate.
 - **Event Schema Evolution:** Changing event structure means coordinating all consumers. Pain point.
 - **Debugging Gets Tricky:** Tracing event flow across the system is harder than following synchronous calls.
 - **Operational Overhead:** Managing Kafka cluster, configuring topics, tuning partitions—it's a lot of work.
 - **Message Ordering:** Making sure events get processed in the right order (UserRegistered before VideoWatched) needs careful design.
 - **Duplicate Processing:** Kafka's at-least-once delivery means consumers need to be idempotent. Extra complexity.
 - **Testing Challenges:** Simulating complex event sequences for integration tests is tough.
 - **Infrastructure Costs:** Event broker, event store, monitoring tools all add up.
-

Comparison and Recommendation

Comparison Matrix

Criteria	Monolithic	Microservices	Event-Driven
Development Speed	5	2	3
Scalability	2	5	4
Fault Isolation	1	4	5
Operational Complexity	5	2	2
Consistency	5	3	2
Design Pattern Fit	4	5	4
Team Coordination	3	5	4
Cost (Initial)	5	2	2

Suitability Analysis

For Our Current POC Stage:

Monolithic is best right now because:

- We've already got all four design patterns (Factory, Strategy, Observer, Singleton) implemented and working together
- Single deployment makes demonstrating everything way simpler during presentations
- Our team of 3 can coordinate easily on a shared codebase
- No infrastructure overhead: just H2 database and one JAR file
- Fast iteration for getting through the remaining milestones

For Production-Scale StreamFlix:

Microservices Architecture is the best choice for the following reasons:

1. Natural Service Boundaries Match Our Design Patterns

Our existing design patterns map cleanly to microservices:

- **Content Service** (Factory Pattern): Manages movie/TV series creation and catalog. Elena could own this.
- **Video Service** (Observer Pattern): Handles watch events and publishes to message queue instead of in-process observers. Daria could lead this.
- **Recommendation Service** (Strategy Pattern): Uses different algorithms based on user state, needs independent scaling. Miruna could focus here.
- **User Service**: Authentication and profiles.
- **Analytics Service**: Processes events for metrics.
- **Notification Service**: Handles alerts.

2. Streaming Platform Requirements Demand Independent Scaling

Video streaming has very different scaling needs:

- **Content browsing** is read-heavy (80% reads, 20% writes) → scale Content Service horizontally with MongoDB
- **Recommendations** are CPU-intensive (algorithm processing) → scale with more powerful instances
- **Video streaming** is bandwidth-heavy → scale with CDN and specialized streaming infrastructure
- **Analytics** can batch process during off-peak hours → different resource profile entirely

With monolithic architecture, we'd have to scale everything together even though only recommendations are hitting CPU limits.

3. Team Autonomy for Our 3-Person Team

Microservices let us work in parallel:

- Alina works on Content Service (Factory pattern, MongoDB integration)
- Miruna develops Recommendation Service (Strategy pattern, Redis caching)
- Daria builds Video Service (Observer pattern, event publishing)

Each person can choose their own:

- Deployment schedule (deploy recommendations without touching video)
- Technology choices (Redis for recommendations, PostgreSQL for users)
- Testing approach (unit tests without mocking 5 other services)

4. Fault Isolation is Critical for Streaming

If recommendations service crashes in a monolith, the entire app goes down—users can't even browse content. With microservices:

- Notification service down? Users still watch videos
- Analytics service processing heavy batch job? Doesn't slow down streaming
- Recommendation service being updated? Content browsing still works

This fault isolation is especially important because we're required to implement this in Milestone 4.

5. Alignment with Industry Standards

Netflix, the platform we're modeling StreamFlix after, uses microservices for exactly these reasons. They have:

- Over 700 microservices
- Each service owned by a specific team
- Independent deployment schedules (1000+ deployments per day)
- Service-specific technology choices

We don't need 700 services, but the same architectural principles apply.

6. Message Queue Integration as Enhancement

In Milestone 5, we'll add Kafka/RabbitMQ for asynchronous communication between microservices. This gives us the best of both worlds:

- **Synchronous REST** for request-response (user login, content browsing)
- **Asynchronous events** for fire-and-forget (watch history, analytics, notifications)

This is better than pure event-driven because:

- Users get immediate response for critical operations (login, search)
- Background tasks run asynchronously without blocking
- We can gradually migrate from REST to events for specific use cases

Why Not Pure Event-Driven?

Event-driven is good, but adds complexity we don't need yet:

- **Eventual consistency** is fine for analytics but not for authentication
- **Event schema evolution** requires coordinating all consumers when we change events
- **Debugging** distributed events is harder than REST calls with correlation IDs
- **Learning curve** is steeper—our team would need to master both microservices AND event streaming

Better to start with microservices + REST, then add events in Milestone 5 for specific async needs.

Why Not Remain Monolithic?

Monolithic simply can't meet StreamFlix's production needs:

- Can't scale recommendations independently from content browsing
- Single point of failure means any bug brings down the entire platform
- All three team members would be constantly stepping on each other's code
- Can't experiment with new technologies (e.g., trying Go for streaming service)
- Deployment risk increases with every change affecting the entire codebase

Final Recommendation

Current Stage (POC):

Continue with **Monolithic Architecture**

- Keeps all four design patterns cleanly demonstrated
- Simplifies presentation and grading
- Fast iteration for completing remaining milestones

Production Implementation (Next Milestone):

Transition to **Microservices Architecture**

Service Decomposition:

1. **User Service** (PostgreSQL) - Authentication, profiles, subscriptions
2. **Content Service** (MongoDB) - Movies/TV series catalog with Factory pattern
3. **Video Service** - Watch events, Observer pattern for event publishing
4. **Recommendation Service** (Redis) - Strategy pattern for personalized recommendations
5. **Analytics Service** - View counts, ratings aggregation
6. **Notification Service** - Email/push notifications

Communication:

- REST APIs for synchronous request-response
- Message queue (Kafka) added in Milestone 5 for async events

Deployment:

- Docker containers for each service
- Kubernetes for orchestration (or Docker Compose for simpler setup)
- API Gateway (Spring Cloud Gateway) for routing

Evolution Path:

1. Start with 3 core services in Milestone 4 (User, Content, Video)
2. Add REST communication between services
3. Implement API Gateway for unified entry point
4. In Milestone 5, integrate Kafka for async communication
5. Gradually expand to 6 services as features grow

This microservices approach gives us the scalability, fault isolation, and team autonomy we need while preserving our design patterns and setting us up for event-driven enhancements in Milestone 5.

Conclusion

Monolithic is perfect for our current proof-of-concept stage as it lets us develop fast and demonstrate our design patterns clearly. For production, microservices architecture gives us the best combination of scalability, fault isolation, and team autonomy while aligning with our project milestones.

StreamFlix - SDT Project

Team: Bilciurescu Elena-Alina, Solomon Miruna-Maria, Toma Daria-Maria

Group: 1241EA CTI-E